Инструкция по проекту: Просмотрщик хронологии СУ УЭЦН

Что это такое

Веб-приложение (один HTML-файл, ~1900 строк) для просмотра бинарных файлов хронологии (.arh) станций управления (СУ) установками электроцентробежных насосов (УЭЦН) производства НПО «Эталон» (серия КСУ-27). Эти станции управляют погружными электродвигателями (ПЭД) для нефтедобычи через частотно-регулируемый привод (ЧРП/инвертор).

Аналог — штатная Windows-программа «Читалка Эталон», но наша реализация работает в браузере без установки.

Что есть в репозитории GitHub

1. `viewer_su_arh_v9.html` — основной HTML-файл приложения (единственный файл, который нужно развивать)
2. `Configs/` — папка с XML-конфигурациями для разных версий контроллеров (27_001.xml … 27_061.xml, а также 25_xxx, 26_xxx, 32_xxx, 40_xxx). Ключевой конфиг для текущей реализации — **27_038.xml**
3. **Два .arh файла** — реальные хронологии для тестирования

Цель проекта

Создать полнофункциональный просмотрщик хронологии СУ УЭЦН в виде одного HTML-файла, который:

- Парсит бинарные .arh файлы хронологии
- Отображает графики всех параметров (токи, напряжения, частота, давление, температура, вибрация и т.д.)
- Ведёт журнал событий с правильной классификацией ПУСК/СТОП/Защита/Уставки
- Работает без бэкенда, полностью в браузере
- В перспективе — поддерживает все версии контроллеров (27_001…27_061), а не только 27_038

Архитектура файла .arh

Общая структура

```
ЗАГОЛОВОК:СЕГМЕНТ0.СЕГМЕНТ1.СЕГМЕНТ2...СЕГМЕНТn
```

- **Заголовок** (до первого `:`): HEX-строка, разделённая `-`, содержит номер куста, скважины, серийный номер СУ
- **Сегменты** разделены `.` и `:` (оба трактуются как разделитель)

- **Самый длинный сегмент** — это основные данные хронологии (измерения + состояние)
- Другие сегменты: пусковые характеристики, настройки, и т.д.

Структура записи хронологии

Каждая запись — HEX-строка фиксированной длины:

```
[20 символов заголовка] + [nParams × 4 символа параметров]
```

Заголовок записи (20 hex chars = 10 байт):

```
Байты 0-3: неизвестно (всегда 0000001a в текущих файлах)
Байт 4: месяц (hex)
Байт 5: день (hex)
Байт 6: неизвестно
Байт 7: час (hex)
Байт 8: минута (hex)
Байт 9: секунда (hex)
```

Параметры: каждый — 4 hex chars = 16-bit unsigned integer (uint16).

Определение nParams

nParams определяется подбором: перебираем значения от 26 до 100, проверяя делимость длины основного сегмента на `(20 + nParams × 4)`. Первое совпадение — верный nParams. Типичные значения: 26, 29, 34, 35, 36, 37.

Параметр «Состояние» (последний параметр)

Всегда последний в массиве параметров (индекс `nParams - 1`). Содержит uint16:

- **Lo-byte** (`raw & 0xFF`): код состояния из ENUM3 (см. ниже)
- **Hi-byte** (`(raw >> 8) & 0xFF`): обычно 0

Параметр «Вращение» / P0 (первый параметр)

Всегда первый (индекс 0). Формат format_id=11. При обычной работе `P0.lo` совпадает с `State.lo`. **Но в момент переходов** P0.lo содержит:

- Код причины ПУСК из ENUM2 (при переходе State.lo → 0)
- Код причины СТОП из ENUM3 (при переходе State.lo → 234)
- Код системного события из ENUM26 (при любом State)

Записи FFFE (0xFFFE = 65534)

Когда `State = 0xFFFE`, это **не запись измерений**, а сообщение:

- `params[0]` hi-byte = источник: `0x90` = Дистанционно, `0x98` = Оператор, `0xE0` = Система
- `params[1]` = индекс уставки (из enum 0 XML-конфигурации)
- `params[2]` = старое значение
- `params[3]` = новое значение
- Остальные params = 0xFFFE (не используются)

Если `params[0] = 0x4800` и все остальные FFFE — это пустое системное событие (пропускаем).

Ключевые перечисления (ENUM)

ENUM3 — Состояния и причины отключения

Диапазон	Значение
1–89	Коды аварий/защит (Недогрузка, Перегрузка, КЗ фаз, и т.д.)
234	Останов (промежуточное состояние при СТОП)
235–247	Рабочие режимы (Токоограничение=235, Разгон=242, Вывод на реж.=243, и т.д.)
251	Простой
252	Работа
253	Пауза
254	Ожидание АПВ
255	Блокировка

ENUM2 — Причины включения

Код	Значение
35	Автоматическое включение
36	Оператор
37	Дистанционное включение
49	Запрет пуска

ENUM26 — Системные сообщения

Код	Значение
77	Деблокировка контроллера
78	Смена переключателя работа/стоп
79	Подключен ТМС
81	Считывание хронологии
82	Открыта дверь
85	Подключение СЭЭ
93	Стоп по Modbus
94	Пуск по Modbus
95	Стоп с дискр. входа
96	Пуск с дискр. входа
105	Дист. деблокировка
…	(полный список в HTML-файле)

Алгоритм определения ПУСК/СТОП

Это самая сложная и коварная часть. Алгоритм основан на отслеживании переходов State.lo между категориями состояний, с чтением причины из P0.lo.

Категории состояний

```javascript
isIdle(lo):      lo ∈ {251, 253, 254, 255}  // Простой, Пауза, Ож.АПВ, Блокировка
isRunning(lo):   lo === 252                 // Работа
isStarting(lo):  235 ≤ lo ≤ 247             // Разгон, Вывод на реж., и т.д.
isOstanov(lo):   lo === 234                 // Останов (переходное)
isFault(lo):     1 ≤ lo ≤ 89                // Аварии/защиты
isActiveWork(lo): isRunning(lo) || isStarting(lo)
```

КРИТИЧЕСКИ ВАЖНО: `234 (Останов)` НЕ входит в `isStarting`! Это вызвало дубликаты СТОП.

Паттерны ПУСК

Паттерн 1 (основной): `State.lo` переходит в `0` из idle/fault состояния

- P0.lo проверяется:
  - Если P0.lo ∈ ENUM2 (35,36,37,49) → **ПУСК** с причиной из ENUM2
  - Если P0.lo ∈ {94, 96} → **ПУСК** по Modbus / с дискр. входа
  - Если P0.lo ∈ ENUM26 → **Системное событие** (НЕ ПУСК!) — «Деблокировка контроллера» и т.п.
  - Иначе → лог-запись

Паттерн 2: `State.lo` переходит напрямую в Starting (235-247) из idle, **только если prevLo ≠ 0** (чтобы не дублировать Паттерн 1)

Паттерны СТОП

Паттерн 1: `State.lo` переходит в `234` (Останов) из activeWork

- P0.lo на этой записи содержит причину из ENUM3 (33=Программный останов, 34=Оператор, и т.д.)

Паттерн 2: `State.lo` переходит в fault (1-89) из activeWork

- Сам код аварии = причина

Паттерн 3: `State.lo` переходит в idle (254=Ож.АПВ и др.) из activeWork, **только если prevLo ≠ 234** (чтобы не дублировать Паттерн 1)

- Причина ищется в P0.lo ±3 записей от точки перехода

Переменные отслеживания

```javascript
prevLo     // State.lo предыдущей записи
prevRealLo // State.lo последней записи, где lo ≠ 0 (для «пробития» через код 0)
effectivePrev = (prevLo === 0) ? prevRealLo : prevLo
```

`effectivePrev` нужен потому что State часто проходит через промежуточное значение `0` при переходах. Например: `Работа(252) → 0 → Останов(234)`. Без `effectivePrev` мы бы потеряли информацию о том, что до Останова была Работа.

ENUM26 Overlay

После классификации ПУСК/СТОП, для записей, которые НЕ были классифицированы как start/stop, проверяется: если P0.lo содержит код из ENUM26 и P0.lo ≠ State.lo, то label записи заменяется на текст из ENUM26. Это позволяет показывать «Деблокировка контроллера» и т.п. во время состояний Блокировка(255) или ОжАПВ(254).

Форматирование значений параметров

Каждый параметр имеет `format_id` из XML-конфигурации:

format_id	Формула	Пример
0	raw	395
1	raw / 10	34.4
2	raw / 100	0.85
3	raw / 1000	6.000
4	sign bit 15 + (raw & 0x7FFF) / 10	Знаковая температура
11	raw (enum lookup)	Состояние, Вращение
23	(raw >> 8) & 0xFF	Температура IGBT
37	(raw >> 8) & 0xFF	Температура фильтра

Конфигурации параметров (PCFG)

В HTML захардкожены конфигурации для nParams = 26, 29, 34, 35, 36, 37. Каждая определяет: имя, format_id, группу (для графика), цвет. Конфигурации взяты из XML-файлов (mixed_block и launches_block).

При неизвестном nParams — берётся ближайшая конфигурация и дополняется generic параметрами.

Подводные камни (КРИТИЧЕСКИ ВАЖНО)

1. Код 234 (Останов) — НЕ «рабочий режим»

Ошибка: изначально `isStarting` определялся как `234 ≤ lo ≤ 247`, включая Останов. Это приводило к тому, что при последовательности `Работа → Останов → Пауза` запись Пауза тоже определялась как СТОП (потому что Останов(234) считался «active»), дублируя событие. **Решение:** `isStarting` = `235 ≤ lo ≤ 247` (Останов исключён).

2. Промежуточный код 0 при переходах

Проблема: Контроллер часто записывает промежуточную запись с State.lo=0 между состояниями: `Простой → 0 → Разгон` или `Работа → 0 → Останов`. Если `prevLo` обновляется на 0, мы теряем контекст предыдущего реального состояния. **Решение:** переменная `prevRealLo` хранит последний ненулевой State.lo, а `effectivePrev` «пробивает» через ноль.

3. Причина ПУСК/СТОП — в параметре P0, не в State

Ошибка: изначально причина искалась в hi-byte параметра State. На самом деле причина закодирована в

lo-byte параметра P0 («Вращение», индекс 0) в момент перехода. **Как устроено:**

- При ПУСК: запись с State.lo=0 содержит P0.lo = код из ENUM2 (причина включения)
- При СТОП: запись с State.lo=234 содержит P0.lo = код из ENUM3 (причина отключения)

4. P0.lo=35 — омоним (есть и в ENUM2, и в ENUM3)

ENUM2[35] = «автоматическое включение», ENUM3[35] = «Дистанционное отключение». Различаем по контексту (направление перехода State).

5. ENUM26 коды в P0.lo при State.lo=0 — это НЕ ПУСК

Ошибка: при State.lo=0 и P0.lo=77 (Деблокировка контроллера) код ошибочно определялся как ПУСК. На самом деле коды ENUM26 (77, 78, 79, 81, 82, 85 и др.) — это системные события, не связанные с пуском. **Исключения:** P0.lo=94 (Пуск по Modbus) и P0.lo=96 (Пуск с дискр. входа) — это реальные ПУСК из ENUM26.

6. Дубликаты ПУСК при последовательности code-0 → Разгон

Проблема: `Простой → 0 (ПУСК) → Разгон`. Паттерн 1 ловит ПУСК на записи с code-0. Затем Паттерн 2 тоже ловит ПУСК на записи Разгон (потому что effectivePrev=Простой, idle). **Решение:** Паттерн 2 срабатывает только если `prevLo ≠ 0`.

7. Дубликаты СТОП при последовательности Останов → Пауза

Проблема: `Работа → Останов (СТОП) → Пауза`. Паттерн 1 ловит СТОП на Останов. Затем Паттерн 3 ловит СТОП на Пауза (потому что Останов ∈ isStarting). **Решение:** (а) Останов исключён из isStarting, (б) Паттерн 3 не срабатывает если `prevLo === 234`.

8. FFFE записи и Web Worker

Проблема: при парсинге FFFE записей внутри Web Worker, функция `fmtSettingVal` была определена **внутри цикла for** через `function declaration`. Это вызывало ошибку в strict-mode Web Worker, из-за чего worker падал при первом FFFE-записе и ВСЕ последующие события не обрабатывались. **Решение:** вынести все функции из цикла for наружу.

9. Год в timestamps

Файл .arh не содержит год в записях (только месяц, день, час, минута, секунда). Год извлекается из имени файла (паттерн `2026_02_15`). При переходе декабрь→январь — год автоматически инкрементируется.

10. Определение семантических индексов параметров

Параметры определяются по имени (нечёткий поиск). Например, «Fт,Гц» — турбинная частота — ищется по `nl.startsWith('fr')`. Проблема: названия могут отличаться между конфигурациями (27_038 vs 27_044). Нужно проверять на разных конфигурациях.

11. Разная нумерация записей

Оригинальная читалка использует **1-based** нумерацию (первая запись = 000001). Наш HTML использует

0-based (первая запись = 000000). Это сдвигает все номера на 1.

Что реализовано

- [x] Парсинг .arh файла (заголовок, определение nParams, декодирование timestamps)
- [x] Графики с min/max downsampling (все пики видны при любом масштабе)
- [x] Zoom/Pan (колесо мыши, перетаскивание, кнопки)
- [x] Раздельный и наложенный режимы отображения графиков
- [x] Журнал событий с виртуальным скроллом (100k+ записей)
- [x] Классификация ПУСК/СТОП с причинами из P0.lo
- [x] Системные события ENUM26 (Деблокировка, Смена переключателя, и т.д.)
- [x] Парсинг FFFE записей (изменение уставок с форматированием значений)
- [x] Фильтры журнала (Все, ПУСК/СТОП, Защиты, Уставки, Работа, Прочие)
- [x] Двойной клик на графике → переход в журнал на ближайшее событие
- [x] Resize панелей графиков (drag)
- [x] Конфигурации для nParams = 26, 29, 34, 35, 36, 37

Что НЕ реализовано (TODO)

- [ ] Поддержка всех XML-конфигураций (сейчас хардкод для 27_038)
- [ ] Автоопределение конфигурации по серийному номеру СУ
- [ ] Парсинг сегментов пусковых характеристик
- [ ] Отображение параметра «Вращение» (ПРЯМОЕ/ОБРАТНОЕ) в журнале
- [ ] 1-based нумерация записей (как в оригинальной читалке)
- [ ] Экспорт данных в CSV/Excel
- [ ] Печать графиков/журнала
- [ ] Парсинг всех сегментов .arh (сейчас только основной + FFFE)
- [ ] Уставки, которых нет в SETTING_NAMES, показываются как «Уставка NNN»
- [ ] Поддержка полного списка VAL_ENUMS (сейчас только основные: 6,7,14,15,16,17)
- [ ] Парсинг format_id=6 (часы:минуты) и format_id=7 (минуты:секунды) для уставок
- [ ] Тёмная/светлая тема
- [ ] Мобильная адаптация

Как тестировать

1. Открыть HTML-файл в браузере
2. Перетащить .arh файл на страницу или нажать для выбора
3. Проверить графики: sidebar → включить/выключить параметры
4. Открыть журнал событий (кнопка «Журнал событий» в шапке)
5. Сравнить с оригинальной читалкой:
    - ПУСК должен показывать причину (автоматическое/оператор/дистанционное)
    - СТОП должен показывать причину (Программный останов/Оператор/защита)

- Изменения уставок должны показывать имя, старое и новое значение
- Системные события (Деблокировка, Пуск по Modbus) должны отображаться
- Нет дубликатов ПУСК/СТОП

Структура XML-конфигурации (для справки)

```xml
<!-- Перечисления -->
<enum index="0" name="Уставки">...</enum>       <!-- Имена уставок -->
<enum index="2" name="Причины включения">...</enum>  <!-- ENUM2 -->
<enum index="3" name="Причины отключения">...</enum> <!-- ENUM3 -->
<enum index="26" name="Системные сообщения">...</enum> <!-- ENUM26 -->

<!-- Блоки данных -->
<settings_block block_position="0">...</settings_block>  <!-- Формат уставок -->
<mixed_block type="4" block_position="1"                 <!-- Параметры измерений -->
 message_enum_id="26" on_enum_id="2" off_enum_id="3">
 <parameter>
  <position>N</position>
  <n>Имя параметра</n>
  <format_id>FMT</format_id>
  <enum_id>ENUM</enum_id>
 </parameter>
 ...
</mixed_block>
<launches_block type="4" block_position="2" ...>         <!-- Пусковые характеристики -->
```